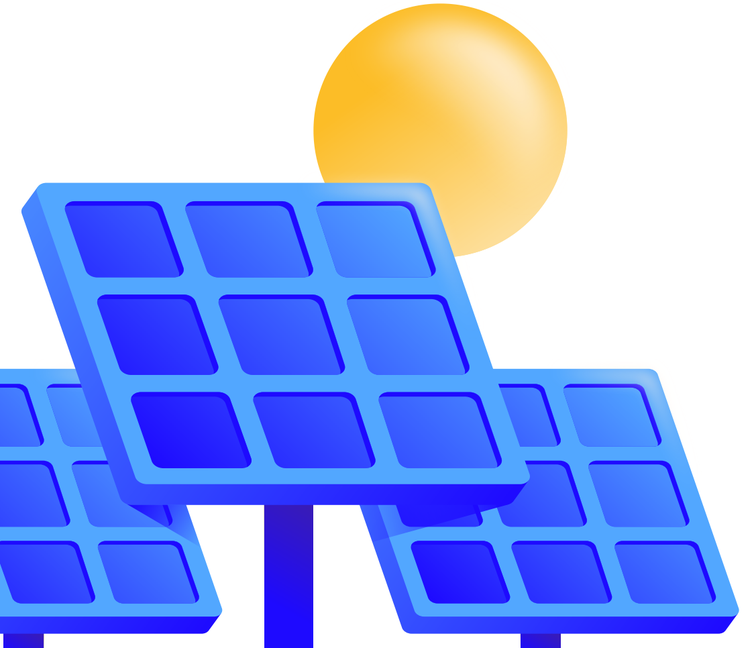




SOLAR POINTS

Apresentação





Equipe

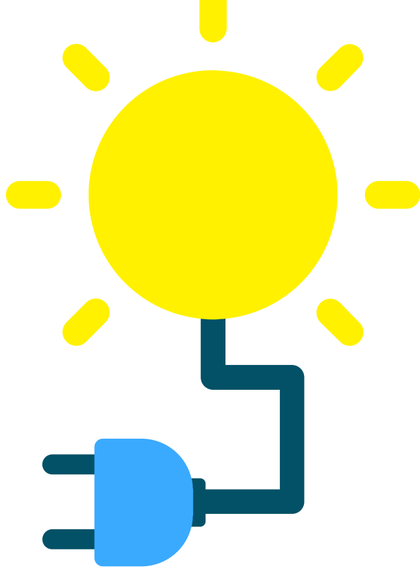
Felipe Grolla Freitas

Guilherme Oliveira Nogueira

Giovanna Lima Salvagnini

Henrique Spadaccia Chambó





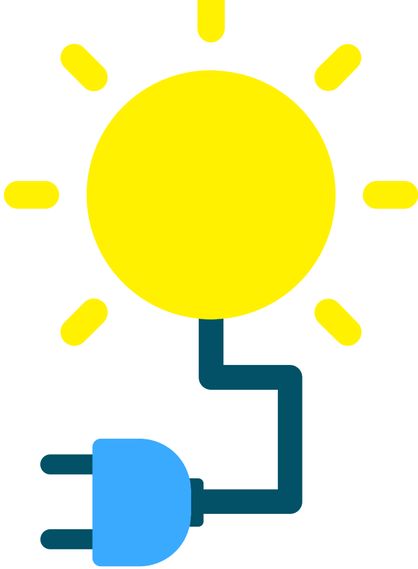
Introdução

- **Problemática:**
  - Muitos espaços públicos permanecem sem iluminação noturna (pontos de ônibus, vielas, praças, áreas remotas).
  - Instalação de redes elétricas nesses locais é cara e complexa.
  - Falta de iluminação gera insegurança, desconforto e baixa acessibilidade.
- **Nossa solução:**
  - Sistema autônomo de iluminação alimentado por energia solar.
  - Focado em segurança, acessibilidade e sustentabilidade.
  - Alternativa viável para locais sem infraestrutura elétrica.









Objetivo do projeto

- **Propósito Central**
  - Desenvolver um sistema de iluminação autônomo e inteligente para locais públicos sem infraestrutura elétrica.
- **Metas Específicas**
  - Proporcionar segurança e bem-estar à população.
  - Garantir baixo custo e sustentabilidade ambiental.
  - Monitorar o fluxo de pessoas em tempo real como apoio à segurança pública.
  - Demonstrar a viabilidade da eletrônica e microcontroladores (ESP32) em soluções práticas.

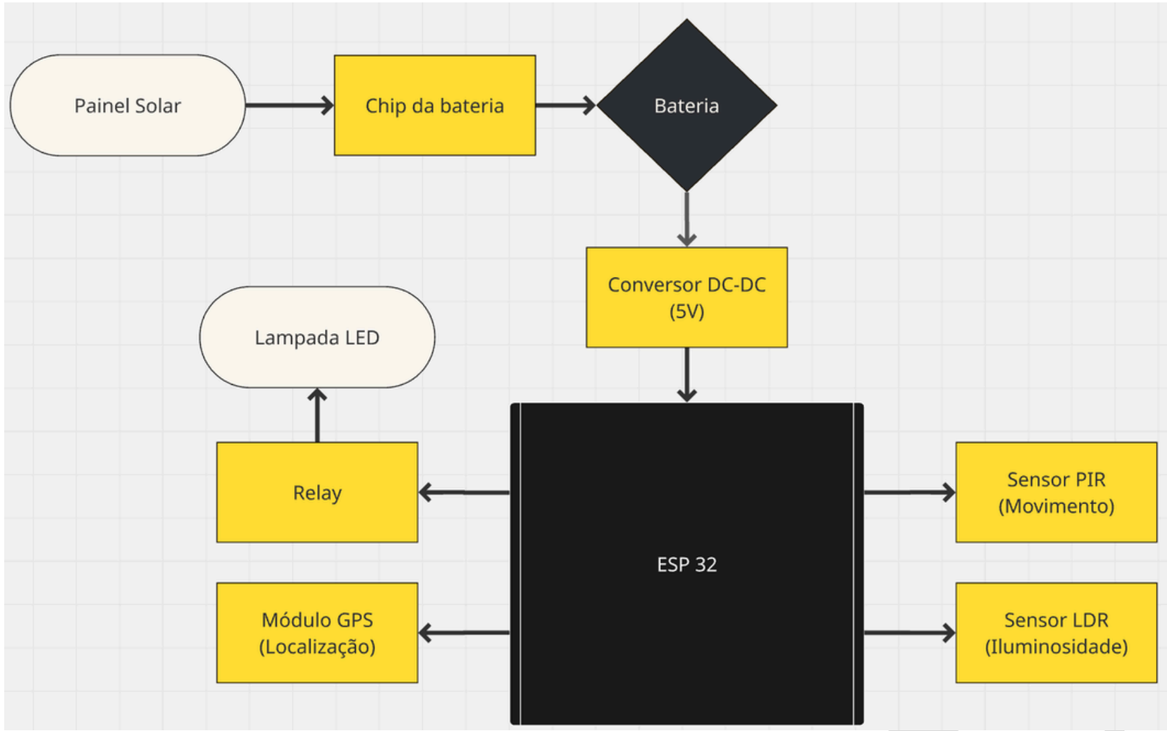




Desenvolvimento de Hardware

- **Estratégia de Projeto**
  - Foco em eficiência energética, segurança, robustez e modularidade.
  - Arquitetura com separação de baixa (5V) e alta tensão (12V) → proteção dos circuitos lógicos.
- **Fontes e Gerenciamento**
  - Painel Solar (12V) + Bateria de Lítio 3S com BMS.
  - Chip de gerenciamento de energia (Power Path).
  - Conversor DC-DC Step-down → alimentação estável de 5V.
- **Lógica e Sensores**
  - ESP32: controle central, Wi-Fi e Bluetooth.
  - Sensores: PIR (movimento) + LDR (luminosidade).
  - Módulo GPS: geolocalização e monitoramento de pontos.
- **Atuação e Segurança**
  - Relé 5V com optoacoplador → isola lógica de potência.

Fluxo de Operação



Arquitetura Geral do Circuito

Esquemático do circuito

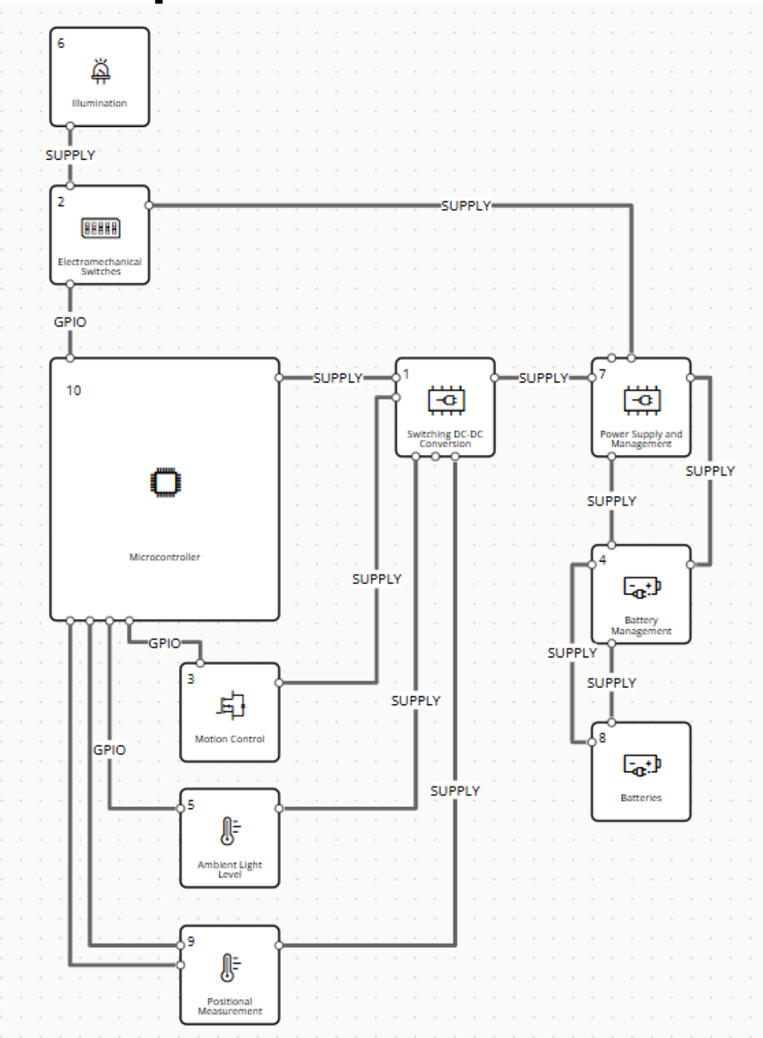
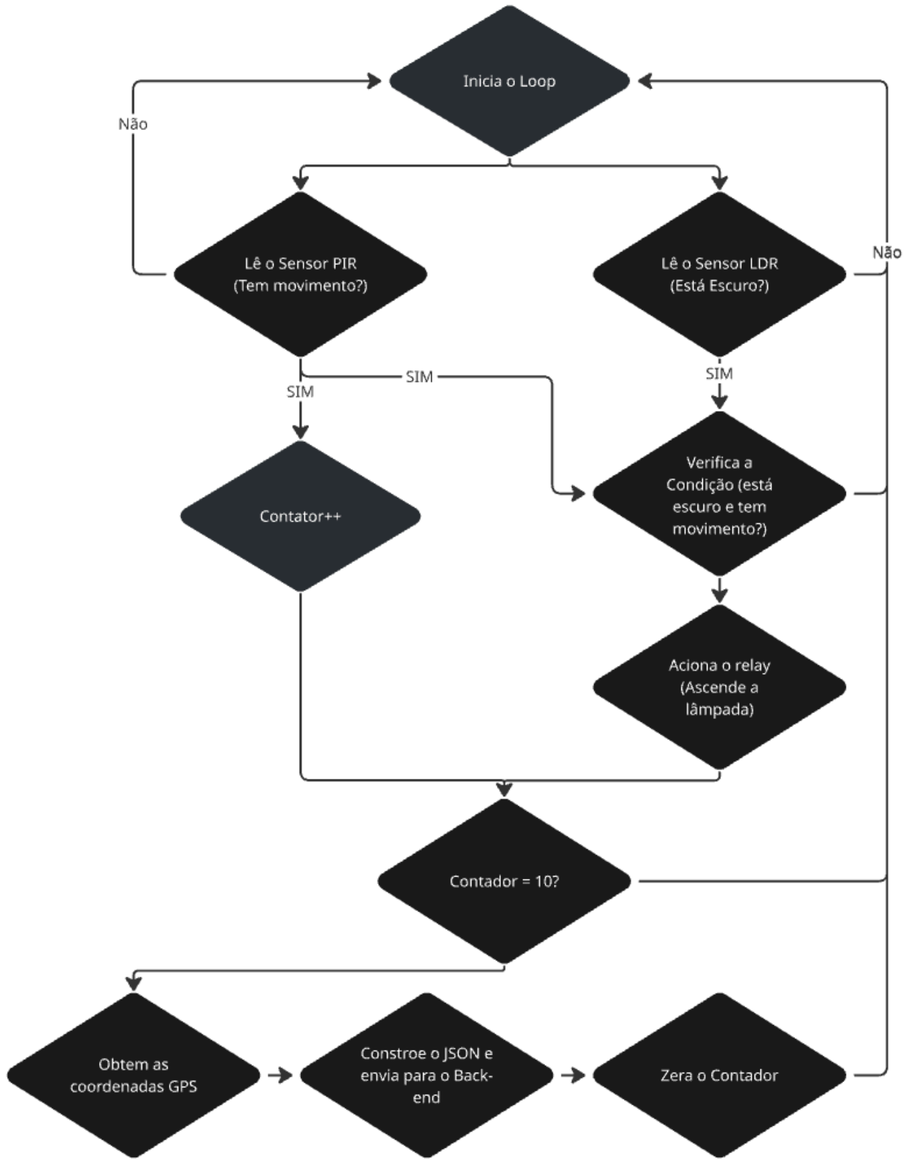


Imagem real do circuito

Desenvolvimento do Software

Programação e Software do ESP32

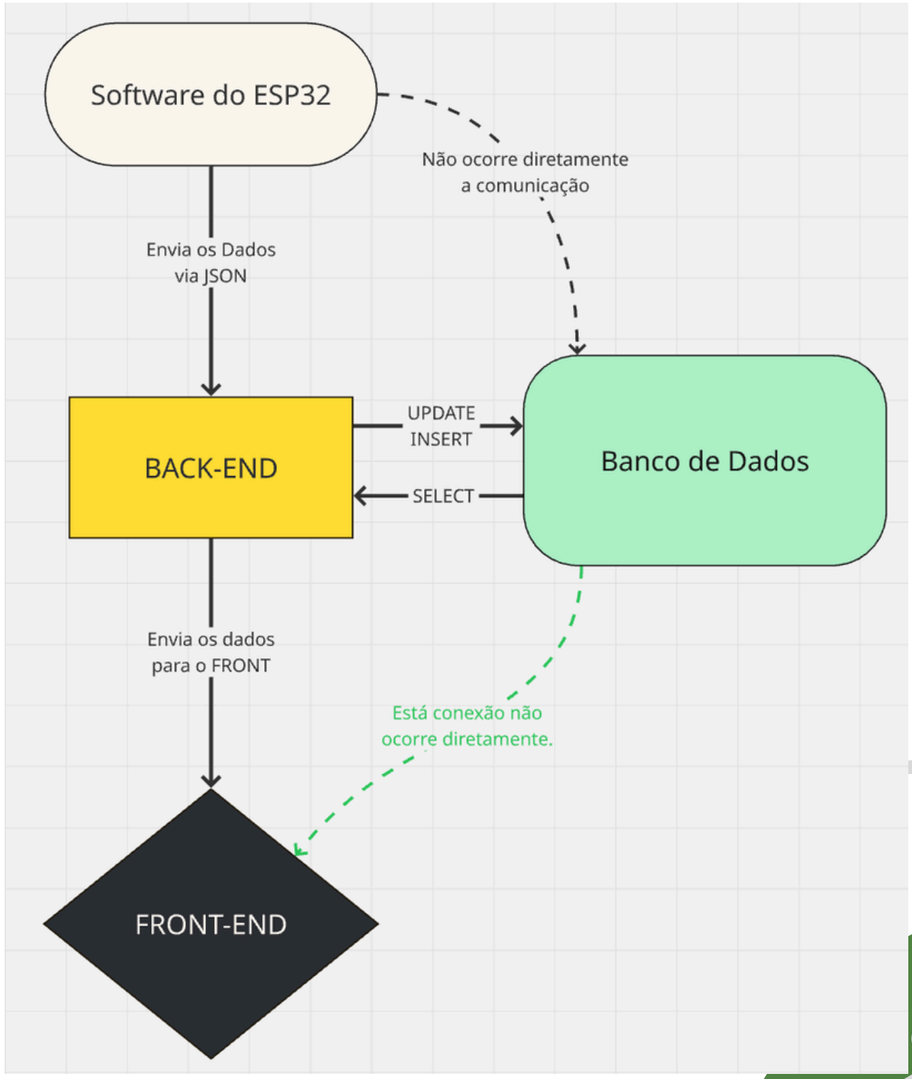
- Lógica Operacional (loop)
  - Movimento: contagem com borda de subida do PIR
  - Luminosidade: leitura contínua do LDR (limiar = 1500)
  - Relé: acionado somente se (movimento == true) e (ambiente escuro)
- Coleta e Comunicação
  - Envio de dados a cada 10 movimentos
  - Obtenção de coordenadas via módulo GPS
  - Payload JSON → Requisição HTTP POST para API
  - Após envio: contador resetado, iniciando novo ciclo



Desenvolvimento do Software

Servidor Back-end

- Linguagem da programação e estrutura
  - Python com Framework do Flask
  - Implantação na Vercel (serverless hosting)
- Estrutura e funcionamento da Aplicação (main.py)
  - /receber-dados-esp (POST)
    - Recebe JSON do ESP32 (movimentos + coordenadas GPS)
    - Arredonda coordenadas (6 casas decimais)
    - Lógica UPSERT (UPDATE se coordenadas já existem, INSERT se não existem)
  - /api/items (GET)
    - Consulta registros no banco (ID, contador, latitude, longitude)
    - Retorno em JSON para consumo no front-end

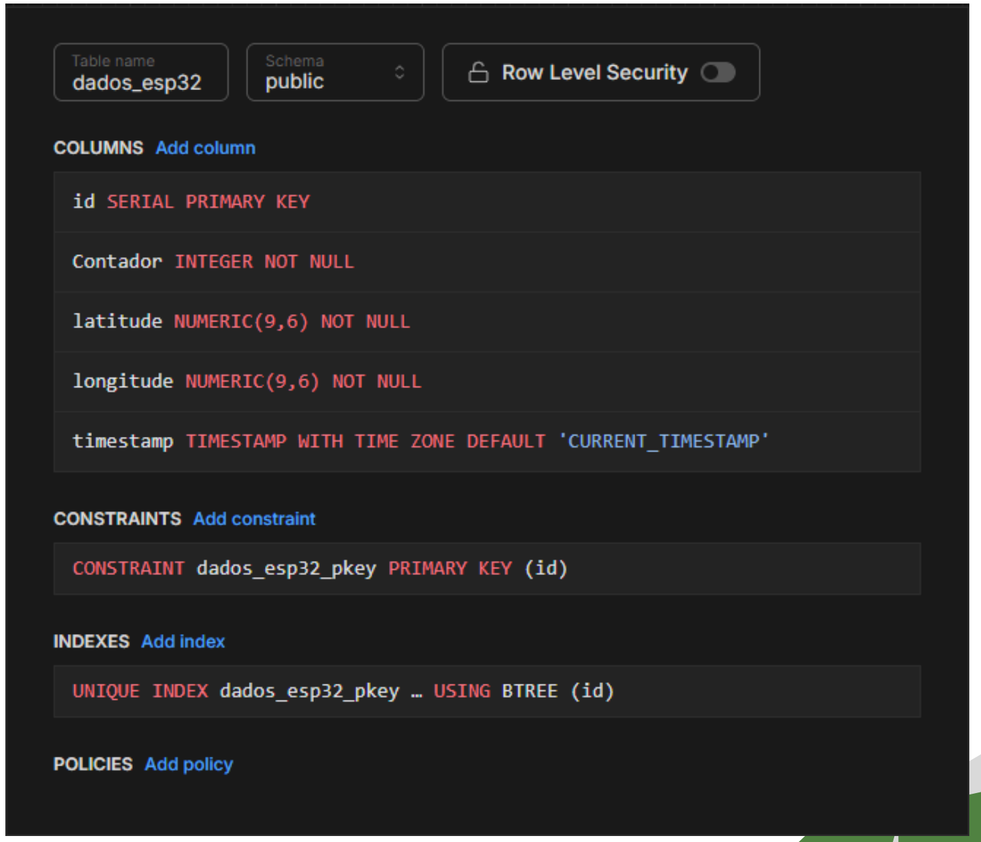


Desenvolvimento do Software

Banco de Dados em Nuvem

- Plataforma:
  - NeonDB (PostgreSQL serverless)
  - Integração nativa com Vercel
- Estrutura da Tabela (dados_esp32):
  - id → SERIAL (PK, auto-incremento)
  - contador → INTEGER (detecções acumuladas)
  - latitude / longitude → NUMERIC(9,6) (alta precisão)
  - timestamp → TIMESTAMP with time zone (última atualização)
- Comunicação e Lógica:
  - Conexão via DATABASE_URL (variável de ambiente)
  - SELECT → verifica se coordenadas já existem
  - UPDATE → soma valor ao contador existente
  - INSERT → cria novo registro se não existir

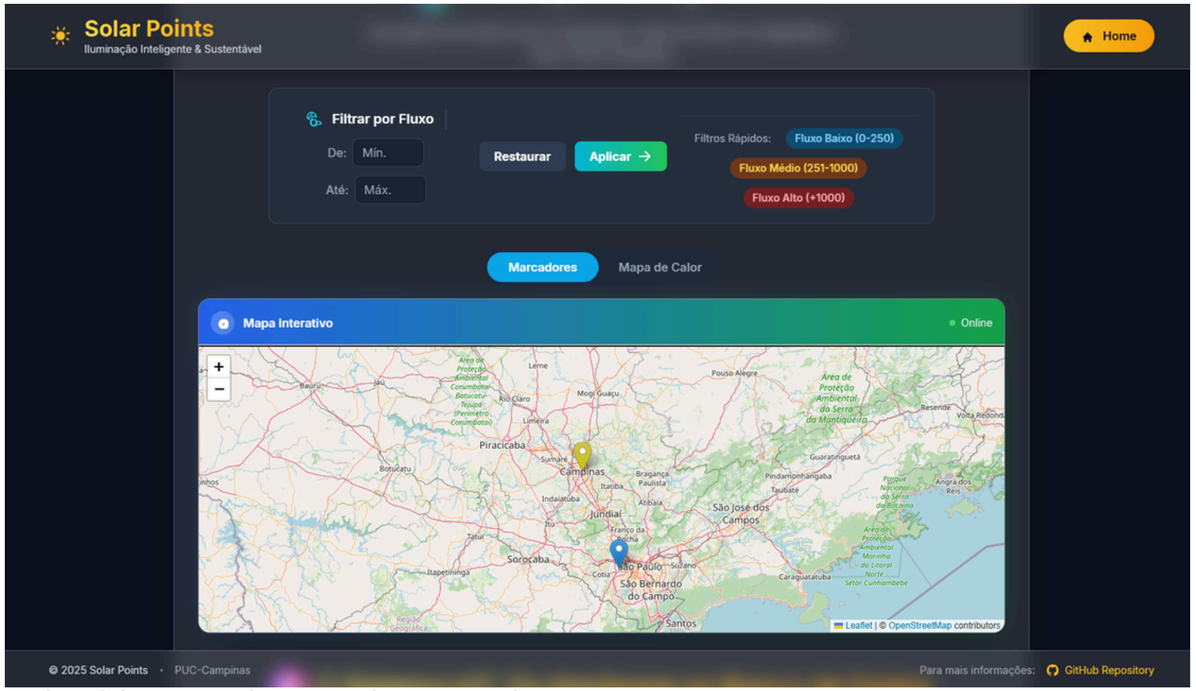




Desenvolvimento do Software

Servidor Front-end

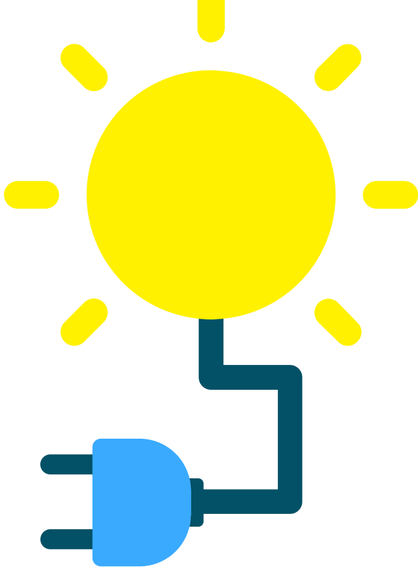
- Tecnologia e Hospedagem:
  - Python com Framework do Flask
  - HTML5, JavaScript (JS) e TailWind (CSS)
  - Hospedado no Vercel
- Lógica do Servidor (BFF - Back for Front):
  - Rota / recebe requisição do usuário
  - Faz GET para a API do back-end (lista de dispositivos)
  - Aplica filtros (ex.: minFlow, maxFlow)
  - Renderiza index.html com dados já filtrados
- Interface do Usuário (Client-Side):
  - Estrutura: HTML + TailwindCSS
  - Interatividade:
    - Maps_API.js → desenha pontos no mapa com base nos dados filtrados
    - Filtro.js → gerencia entradas do usuário e aplica filtros dinâmicos



Link ref: https://solarpointsfront.vercel.app



Conclusões



✅ **Resultados Alcançados**

- Sistema de iluminação autônomo, sustentável e de baixo custo.
- Evolução de conceito analógico → IoT completa e escalável.
- Hardware robusto (energia solar + ESP32 + sensores + GPS).
- Software integrado: C++ (firmware) + Flask (backend) + PostgreSQL + front-end web interativo.

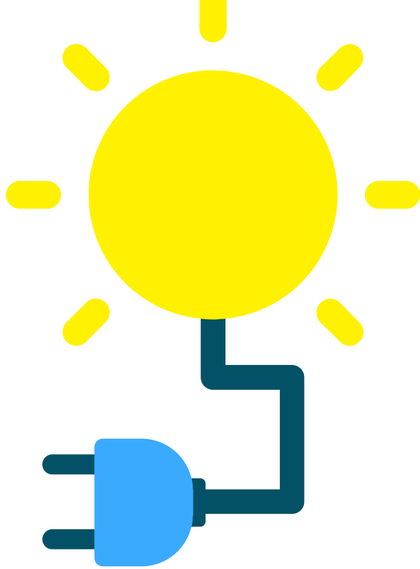
🌍 **Impacto e Viabilidade**

- Redução de custos (infraestrutura + energia elétrica).
- Baixo TCO e ROI favorável.
- Impacto social, urbano e ambiental positivo.









Perguntas?









OBRIGADO



